TEKNİK MİMARİ RAPORU

Anlaşamıyoruz — Tartışma Arabulucu Platformu

Backend: ASP.NET Core · Frontend: Bağımsız Teknoloji

Hazırlayan: Mimari Ekip · Versiyon: 1.0 · Tarih: Mart 2026

Yönetici Özeti

Bu rapor, "Anlaşamıyoruz" tartışma arabulucu platformunun teknik mimarisini, backend ve frontend bileşenlerinin nasıl ayrıştırıldığını, API sözleşmelerini ve geliştirme yol haritasını kapsamaktadır. Platform, iki taraflı görüş girişini alarak yapay zeka destekli derin analiz gerçekleştiren, duygusal ton tespiti yapan ve görsel Venn diyagramı ile çözüm takip listesi sunan bir web uygulamasıdır.

Backend Teknolojisi	ASP.NET Core 8.0 Web API
Frontend Teknolojisi	Ekip tercihi (React / Vue / Angular / Svelte)
AI Entegrasyonu	Anthropic Claude API (claude-sonnet)
İletişim Protokolü	RESTful JSON API + WebSocket (opsiyonel)
Deployment Hedefi	Azure App Service + Azure Static Web Apps
Proje Durumu	Tasarım & Planlama Aşaması — v1.0 hedefi Q2 2026

© 2026 Anlaşamıyoruz Platformu

Genel Mimari

Sistem, frontend ve backend'in tamamen birbirinden bağımsız olduğu bir "Ayrık Mimari" (Decoupled Architecture) prensibine dayanmaktadır. Frontend herhangi bir teknoloji ile yazılabilir; backend ise yalnızca iyi tanımlanmış HTTP endpoint'leri aracılığıyla hizmet verir.

Katmanlar

Katman	Teknoloji	Sorumluluk
Presentation (Frontend)	Ekip tercihi	UI render, kullanıcı girişi, görsel Venn diyagramı
API Gateway	ASP.NET Core 8 Web API	HTTP routing, auth middleware, rate limiting
Application Layer	MediatR + CQRS	İş mantığı, komut/sorgu ayrımı
AI Service Layer	Anthropic SDK (C#)	Claude API çağrıları, prompt yönetimi
Domain Layer	C# Domain Models	İş kuralları, entity sınıfları
Data Layer	Entity Framework Core	PostgreSQL veritabanı erişimi
Cache Layer	Redis	Oturum ve AI yanıt önbelleği

© 2026 Anlaşamıyoruz Platformu

Backend Mimarisi — ASP.NET Core

Proje Yapısı

Önerilen Klasör Yapısı
Anlasalamiyoruz.API / Controllers, Middleware, Extensions Anlasalamiyoruz.Application / Commands, Queries, Services, DTOs Anlasalamiyoruz.Domain / Entities, ValueObjects, Enums Anlasalamiyoruz.Infrastructure / AI, Persistence, Cache, External

Kullanılan NuGet Paketleri

Paket	Versiyon	Kullanım Amacı
Anthropic.SDK	3.x	Claude API entegrasyonu
MediatR	12.x	CQRS pattern, command/query bus
FluentValidation	11.x	Giriş doğrulama
Microsoft.EntityFrameworkCore	8.x	ORM — PostgreSQL erişimi
StackExchange.Redis	2.x	Önbellekleme ve oturum yönetimi
Serilog	3.x	Yapılandırılmış loglama
Swashbuckle (Swagger)	6.x	API dokümantasyonu
Microsoft.AspNetCore.Authentication.JwtBearer	8.x	JWT kimlik doğrulama
Polly	8.x	Retry & circuit breaker pattern

© 2026 Anlaşamıyoruz Platformu

API Endpoint Sözleşmesi

Tartışma (Debate) Endpoint'leri

Method	Endpoint	Açıklama	Request / Response
POST	`/api/v1/debate/start`	Yeni oturum başlat, görüşleri gönder	`{ topic, person1, person2, view1, view2 }`
GET	`/api/v1/debate/{id}`	Mevcut oturumu getir	→ `DebateSessionDto`
POST	`/api/v1/debate/{id}/clarify`	Derinleştirici sorular üret	→ `{ questions: [...] }`
POST	`/api/v1/debate/{id}/analyze`	Tam analizi başlat (async)	→ `202 Accepted + jobId`
GET	`/api/v1/debate/{id}/result`	Analiz sonucunu getir	→ `AnalysisResultDto`
PATCH	`/api/v1/debate/{id}/steps`	Çözüm adımını işaretle	`{ stepIndex, completed: bool }`

Örnek Request / Response

POST /api/v1/debate/start — Request Body

{ "topic": "Şehir dışına taşınmak", "person1": { "name": "Ayşe", "view": "Kariyer için İstanbul'da kalmamız gerekiyor..." }, "person2": { "name": "Mert", "view": "Yaşam kalitesi için daha sakin bir şehre geçelim..." } }

GET /api/v1/debate/{id}/result — Response Body

{ "debateId": "abc-123", "status": "completed", "emotions": { "person1": ["frustration","care"], "person2": ["hope","anxiety"] }, "scores": { "person1": 4, "person2": 3 }, "venn": { "leftPoints": [...], "bothPoints": [...], "rightPoints": [...] }, "verdict": "Her iki taraf da...", "suggestion": "İlk adım olarak...", "actionSteps": ["Adım 1", "Adım 2", "Adım 3"] }

© 2026 Anlaşamıyoruz Platformu

Yapay Zeka Entegrasyonu

Backend, Anthropic'in Claude API'sini iki aşamada kullanır:

Aşama 1 — Derinleştirici Sorular

İlk görüşler alındıktan sonra Claude'a her iki taraf için 2-3 kısa, bağlamı derinleştiren soru üretmesi istenir. Bu adım opsiyoneldir; kullanıcı atlayabilir.

Aşama 2 — Derin Analiz

Tüm veriler (ilk görüşler + ek sorulara verilen cevaplar) birleştirilerek tek bir analiz prompt'u oluşturulur. Claude şu çıktıları üretir:

Çıktı Alanı	Tip	Açıklama
emotions1 / emotions2	string[]	Duygusal ton etiketleri (anger, hope, fear...)
score1 / score2	int (1-5)	Geçerli argüman sayısı
leftPoints / rightPoints	string[]	Her tarafa özgü güçlü noktalar
bothPoints	string[]	Ortak zemin maddeleri
verdict	string	Arabulucu değerlendirmesi (3-4 cümle)
suggestion	string	Yapıcı çözüm önerisi
actionSteps	string[]	Somut takip adımları (4 madde)

Güvenlik & Maliyet Kontrol

Rate Limiting	Her IP için dakikada max 10 istek (Polly + Redis)
Token Sınırı	Her prompt max 2000 input token, 800 output token
Prompt Injection Koruması	Input sanitizasyon middleware — script ve komut karakterleri filtrelenir
Retry Policy	Polly ile 3 deneme, exponential backoff
Sonuç Önbellekleme	Aynı içerikli tartışma Redis'te 1 saat önbelleğe alınır

© 2026 Anlaşamıyoruz Platformu

Frontend Entegrasyonu

Frontend ekibi istediği teknolojiyi özgürce seçebilir. Backend yalnızca JSON üretir ve CORS kuralları yapılandırılmış durumdadır. Aşağıda her framework için entegrasyon notları verilmiştir.

Framework Karşılaştırması

Framework	Önerilen Kullanım	Venn Diyagram Çözümü
React + TypeScript	SPA, büyük ekip	D3.js veya custom SVG
Vue 3 + Vite	Orta ölçek, hız odaklı	Vue-chart-3 veya SVG
Angular	Kurumsal, güçlü tip sistemi	ngx-charts
Svelte / SvelteKit	Performans odaklı, küçük ekip	LayerCake + SVG
Next.js (React SSR)	SEO gereksinimi varsa	D3.js + server render

Frontend'in Yapması Gerekenler

- POST /api/v1/debate/start ile oturum aç, dönen debateId'yi kaydet
- Opsiyonel: POST .../clarify ile derinleştirici soruları al ve kullanıcıya göster
- POST .../analyze ile analizi tetikle; 202 gelirse jobId ile polling yap
- GET .../result ile sonucu al; venn verilerini SVG/canvas ile görselleştir
- PATCH .../steps ile checkbox durumunu backend'e bildir (kalıcı kayıt)

CORS Yapılandırması (Backend)

Program.cs — CORS Policy

```
builder.Services.AddCors(options => {   options.AddPolicy("FrontendPolicy", policy => {
policy.WithOrigins("https://your-frontend.com", "http://localhost:5173")          .AllowAnyHeader()
.AllowAnyMethod();   }); });  app.UseCors("FrontendPolicy");
```


© 2026 Anlaşamıyoruz Platformu

Veritabanı Şeması

Tablo	Temel Alanlar	Açıklama
DebateSessions	Id, Topic, CreatedAt, Status, UserId	Her tartışma oturumu
Participants	Id, SessionId, Name, ViewText, Side (P1/P2)	Taraf bilgileri ve görüşler
ClarifyQuestions	Id, SessionId, ForSide, QuestionText, Answer	Derinleştirici soru-cevaplar
AnalysisResults	Id, SessionId, VennJson, VerdictText, SuggestionText, CreatedAt	AI analiz sonuçları
ActionSteps	Id, SessionId, StepText, IsCompleted, CompletedAt	Çözüm adımları takibi
EmotionTags	Id, SessionId, Side, EmotionKey	Duygusal ton etiketleri

Güvenlik Mimarisi

Kimlik Doğrulama	JWT Bearer Token — isteğe bağlı (anonim mod da desteklenir)
HTTPS	Tüm iletişim TLS 1.3 üzerinden şifreli
API Key Yönetimi	Claude API anahtarı Azure Key Vault'ta saklanır, kod içinde bulunmaz
Input Validation	FluentValidation — view metni max 2000 karakter, XSS temizleme
Rate Limiting	IP başına dakikada 10 istek, aylık 1000 analiz limiti
Audit Log	Tüm AI çağrıları Serilog ile yapılandırılmış biçimde loglanır

© 2026 Anlaşamıyoruz Platformu

Deployment & DevOps

Azure Altyapısı

Servis	Kullanım	Tier
Azure App Service	ASP.NET Core API barındırma	B2 (geliştirme) → P1v3 (prod)
Azure Static Web Apps	Frontend statik dosyaları	Free tier yeterli
Azure Database for PostgreSQL	Ana veritabanı	Flexible Server — Burstable B1ms
Azure Cache for Redis	Önbellekleme	C0 (Basic)
Azure Key Vault	API key ve secret yönetimi	Standard
Azure Application Insights	Monitoring & performans	Pay-as-you-go

CI/CD Pipeline

1. GitHub Actions: main branch'e push → otomatik build & test
2. Docker image oluştur → Azure Container Registry'ye push
3. Azure App Service otomatik güncelleme (deployment slot ile sıfır kesinti)
4. Frontend için Vercel veya Azure Static Web Apps otomatik deploy

Geliştirme Yol Haritası

Aşama	Kapsam	Tahmini Süre
Faz 1 — MVP	Temel API, single-turn analiz, Venn çıktısı	3 hafta
Faz 2 — Derinleşme	Multi-turn sorgulama, duygusal analiz, action steps	2 hafta
Faz 3 — Kullanıcı Yönetimi	JWT auth, oturum geçmişi, çözüm takibi	2 hafta
Faz 4 — Performans	Redis önbellekleme, async kuyruk, rate limiting	1 hafta
Faz 5 — Prodüksiyon	Azure deploy, CI/CD, monitoring, load test	1 hafta

İletişim & Sonraki Adımlar

© 2026 Anlaşamıyoruz Platformu

Bu rapor, frontend ve backend ekipleri arasındaki teknik mutabakat belgesi olarak kullanılacaktır. API sözleşmesindeki herhangi bir değişiklik her iki ekibin onayı ile yapılmalıdır.

Swagger UI	Backend çalışırken https://api.domain.com/swagger adresinde canlı dokümantasyon
Postman Collection	Tüm endpoint'lerin test koleksiyonu repo'da /docs klasöründe
Frontend Mock	Backend hazır olmadan önce /docs/mock-responses klasöründeki JSON dosyaları kullanılabilir
Sorular için	GitHub Issues — "api-contract" etiketi ile açın

— Rapor Sonu —

© 2026 Anlaşamıyoruz Platformu